cuDPEFIM

Generated by Doxygen 1.9.8

1 Class Index 1

1.1 Class List . . . 1

2 File Index 3

2.1 File List . . . 3

3 Class Documentation 5

3.1 ArgParser Class Reference . . . 5

3.1.1 Detailed Description . . . 6

3.1.2 Constructor & Destructor Documentation . . . 6

3.1.2.1 ArgParser() . . . 6

3.1.3 Member Function Documentation . . . 6

3.1.3.1 add_argument() . . . 6

3.1.3.2 add_flag() . . . 7

3.1.3.3 flag_set() . . . 7

3.1.3.4 get() . . . 7

3.1.3.5 get_bool() . . . 8

3.1.3.6 get_separator() . . . 9

3.1.3.7 get_uint32() . . . 9

3.1.3.8 get_uint64() . . . 10

3.1.3.9 parse() . . . 11

3.1.3.10 print_help() . . . 11

3.1.3.11 set() . . . 12

3.2 ArgParser::Argument Struct Reference . . . 12

3.2.1 Detailed Description . . . 12

3.2.2 Member Data Documentation . . . 13

3.2.2.1 default_value . . . 13

3.2.2.2 description . . . 13

3.2.2.3 is_flag . . . 13

3.2.2.4 name . . . 13

3.2.2.5 required . . . 13

3.2.2.6 value . . . 13

3.3 built_file Struct Reference . . . 13

3.3.1 Detailed Description . . . 14

3.3.2 Constructor & Destructor Documentation . . . 14

3.3.2.1 built_file() . . . 14

3.3.2.2 ~built_file() . . . 14

3.3.3 Member Data Documentation . . . 15

3.3.3.1 item_to_itemID . . . 15

3.3.3.2 itemID_to_item . . . 15

3.3.3.3 max_transaction_size . . . 15

3.3.3.4 ordered_twu . . . 15

3.3.3.5 ordered_twu_vector . . . 15

3.3.3.6 primary . . . 15
3.3.3.7 total_items . . . 15
3.3.3.8 transaction_count . . . 16
3.3.3.9 transactions . . . 16
3.4 Item Struct Reference . . . 16
3.4.1 Member Data Documentation . . . 16
3.4.1.1 EMPTY_KEY . . . 16
3.4.1.2 EMPTY_UTIL . . . 16
3.4.1.3 key . . . 16
3.4.1.4 util . . . 17
3.5 parsed_file Struct Reference . . . 17
3.5.1 Detailed Description . . . 17
3.5.2 Member Data Documentation . . . 17
3.5.2.1 key_value_pairs . . . 17
3.5.2.2 twu . . . 18
3.6 raw_file Struct Reference . . . 18
3.6.1 Detailed Description . . . 19
3.6.2 Constructor & Destructor Documentation . . . 19
3.6.2.1 raw_file() . . . 19
3.6.3 Member Function Documentation . . . 19
3.6.3.1 close() . . . 19
3.6.4 Member Data Documentation . . . 19
3.6.4.1 data . . . 19
3.6.4.2 fd . . . 19
3.6.4.3 processed_bytes . . . 20
3.6.4.4 retrieved_bytes . . . 20
3.6.4.5 retrieved_indices . . . 20
3.6.4.6 size_bytes . . . 20
3.6.4.7 submitted_bytes . . . 20
3.7 Timer Class Reference . . . 20
3.7.1 Detailed Description . . . 21
3.7.2 Member Function Documentation . . . 21
3.7.2.1 getTimeDifference() . . . 21
3.7.2.2 printRecords() . . . 21
3.7.2.3 recordPoint() . . . 21
3.8 TimeRecord Struct Reference . . . 22
3.8.1 Detailed Description . . . 23
3.8.2 Member Data Documentation . . . 23
3.8.2.1 label . . . 23
3.8.2.2 rss . . . 23
3.8.2.3 timestamp . . . 23

4 File Documentation **25**

4.1 include/args.hpp File Reference . . . 25
4.1.1 Detailed Description . . . 25
4.2 args.hpp . . . 26
4.3 include/build.hpp File Reference . . . 28
4.3.1 Function Documentation . . . 29
4.3.1.1 build_file() . . . 29
4.4 build.hpp . . . 30
4.5 include/copy.cuh File Reference . . . 31
4.6 include/data_types.cuh File Reference . . . 31
4.7 include/data_types.hpp File Reference . . . 31
4.8 data_types.hpp . . . 32
4.9 include/file.hpp File Reference . . . 32
4.9.1 Detailed Description . . . 33
4.9.2 Enumeration Type Documentation . . . 34
4.9.2.1 ParseState . . . 34
4.9.3 Function Documentation . . . 34
4.9.3.1 load_file() . . . 34
4.9.3.2 parse_file() . . . 35
4.10 file.hpp . . . 36
4.11 include/memory.cuh File Reference . . . 37
4.12 include/mine.cuh File Reference . . . 37
4.13 include/stack.cuh File Reference . . . 37
4.14 include/timer.hpp File Reference . . . 37
4.14.1 Detailed Description . . . 38
4.14.2 Function Documentation . . . 38
4.14.2.1 getMemoryRSS() . . . 38
4.15 timer.hpp . . . 39
4.16 include/validate.cuh File Reference . . . 41
4.17 src/build.cpp File Reference . . . 41
4.17.1 Function Documentation . . . 41
4.17.1.1 build_file() . . . 41
4.17.1.2 buildOrderedTWU() . . . 42
4.17.1.3 createItemMapping() . . . 42
4.17.1.4 createPrimary() . . . 43
4.17.1.5 formatTransactions() . . . 43
4.18 src/file.cpp File Reference . . . 44
4.18.1 Function Documentation . . . 44
4.18.1.1 load_file() . . . 44
4.18.1.2 parse_file() . . . 45
4.18.1.3 parse_line() . . . 46

Index 47

Chapter 1

Class Index

1.1 Class List

Here are the classes, structs, unions and interfaces with brief descriptions:

ArgParser
: Simple command-line argument parser 5

ArgParser::Argument
: Represents a single command-line argument or flag 12

built_file
: Structure holding transformed transaction data for mining 13

Item 16

parsed_file
: Holds parsed file contents and associated structures 17

raw_file
: Holds raw file data and associated metadata 18

Timer
: Tracks execution time and memory usage between labeled checkpoints 20

TimeRecord
: Stores timestamp and memory usage for a labeled point in time 22

Chapter 2

File Index

2.1 File List

Here is a list of all files with brief descriptions:

include/args.hpp
Lightweight C++ command-line argument parser . . . 25
include/build.hpp . . . 28
include/copy.cuh . . . 31
include/data_types.cuh . . . 31
include/data_types.hpp . . . 31
include/file.hpp
Provides structures and functions for handling raw and parsed file data, including file loading, parsing, and utility calculations . . . 32
include/memory.cuh . . . 37
include/mine.cuh . . . 37
include/stack.cuh . . . 37
include/timer.hpp
Simple timer and memory profiler for labeled code segments . . . 37
include/validate.cuh . . . 41
src/build.cpp . . . 41
src/file.cpp . . . 44

Chapter 3

Class Documentation

3.1 ArgParser Class Reference

Simple command-line argument parser.

`#include <args.hpp>`

Classes

- struct Argument

  Represents a single command-line argument or flag.

Public Member Functions

- ArgParser (std::string prog_name)

  Constructs an ArgParser with the specified program name.
- void add_argument (const std::string &name, const std::string &desc, bool required=false, std::optional< std::string > default_val=std::nullopt)

  Adds a new argument (not a flag) to the parser.
- void add_flag (const std::string &name, const std::string &desc)

  Adds a new flag (boolean switch) to the parser.
- void parse (int argc, char *argv[])

  Parses the command-line arguments.
- std::string get (const std::string &name)

  Retrieves the value of a named argument.
- bool get_bool (const std::string &name)

  Retrieves the boolean state of a flag.
- uint32_t get_uint32 (const std::string &name)

  Retrieves the value of a named argument as a 32-bit unsigned integer.
- uint64_t get_uint64 (const std::string &name)

  Retrieves the value of a named argument as a 64-bit unsigned integer.
- char get_separator (const std::string &name)

  Retrieves the separator character corresponding to a named argument's value.
- bool flag_set (const std::string &name)

  Checks whether a flag is set.
- void print_help ()

  Prints the help message to stdout.
- void set (const std::string &name, const std::string &value)

  Manually sets the value of an argument.

3.1.1 Detailed Description

Simple command-line argument parser.

The ArgParser class allows you to define and parse both required and optional arguments (with optional default values) and flags. It also automatically generates a help message for display.

3.1.2 Constructor & Destructor Documentation

3.1.2.1 ArgParser()

```
ArgParser::ArgParser (
            std::string prog_name )  [inline], [explicit]
```

Constructs an ArgParser with the specified program name.

Parameters

prog_name	The program name (used in help messages).

Note

Set the program name to something meaningful for better help output.

3.1.3 Member Function Documentation

3.1.3.1 add_argument()

```
void ArgParser::add_argument (
            const std::string & name,
            const std::string & desc,
            bool required = false,
            std::optional< std::string > default_val = std::nullopt )  [inline]
```

Adds a new argument (not a flag) to the parser.

Parameters

name	The name of the argument (e.g., "--input").
desc	A description of the argument.
required	Whether the argument is required (default: false).
default_val	Default value if the argument is not provided (optional).

Note

If required is true and the user does not supply this argument, parsing will fail and the program will exit with an error.

3.1.3.2 add_flag()

```
void ArgParser::add_flag (
            const std::string & name,
            const std::string & desc )  [inline]
```

Adds a new flag (boolean switch) to the parser.

Parameters

name	The name of the flag (e.g., "--verbose").
desc	A description of the flag.

Note

Flags do not require a value. If present, the value is "true"; otherwise, it remains unset (which you can interpret as false).

3.1.3.3 flag_set()

```
bool ArgParser::flag_set (
            const std::string & name )  [inline]
```

Checks whether a flag is set.

Parameters

name	The name of the flag to check.

Returns

True if the flag has been set, false otherwise.

Exceptions

std::runtime_error	If the flag name is unknown.

3.1.3.4 get()

```
std::string ArgParser::get (
            const std::string & name )  [inline]
```

Retrieves the value of a named argument.

Parameters

name	The name of the argument to retrieve.

Returns

The value of the argument as a std::string.

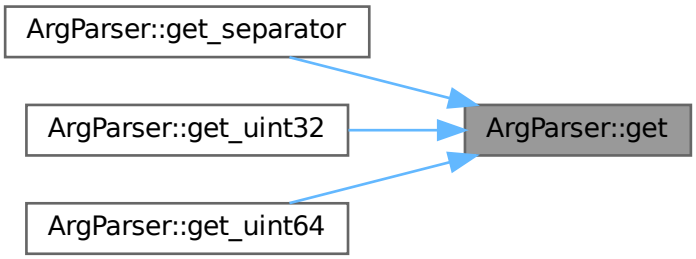
Exceptions

std::runtime_error	If the argument name is unknown.

Note

If the argument was not provided and has no default value, returns an empty string.

Here is the caller graph for this function:



3.1.3.5 get_bool()

```
bool ArgParser::get_bool (
            const std::string & name )  [inline]
```

Retrieves the boolean state of a flag.

Parameters

name	The name of the flag to retrieve.

Returns

True if the flag is present, otherwise false.

Exceptions

std::runtime_error	If the flag name is unknown.

3.1.3.6 get_separator()

```
char ArgParser::get_separator (
            const std::string & name )  [inline]
```

Retrieves the separator character corresponding to a named argument's value.

Parameters

name	The name of the argument that holds the separator string.

Returns

The separator character (e.g., ' ', '\t', ',').

Exceptions

std::runtime_error	If the argument name is unknown, or if the separator string is invalid (which will cause the program to exit).

Here is the call graph for this function:



3.1.3.7 get_uint32()

```
uint32_t ArgParser::get_uint32 (
            const std::string & name )  [inline]
```

Retrieves the value of a named argument as a 32-bit unsigned integer.

Parameters

name	The name of the argument to retrieve.

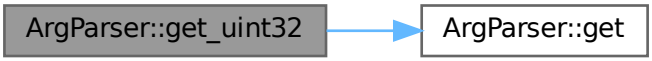
Returns

The argument's value as a uint32_t.

Exceptions

std::runtime_error	If the argument name is unknown or the conversion fails.

Here is the call graph for this function:



3.1.3.8 get_uint64()

```
uint64_t ArgParser::get_uint64 (
            const std::string & name )  [inline]
```

Retrieves the value of a named argument as a 64-bit unsigned integer.

Parameters

name	The name of the argument to retrieve.

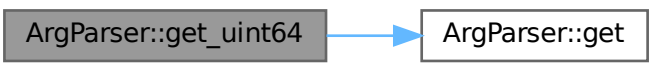
Returns

The argument's value as a uint64_t.

Exceptions

std::runtime_error	If the argument name is unknown or the conversion fails.

Here is the call graph for this function:



3.1.3.9 parse()

```
void ArgParser::parse (
            int argc,
            char * argv[] )  [inline]
```

Parses the command-line arguments.

Parameters

argc	The number of command-line arguments.
argv	The array of command-line argument strings.

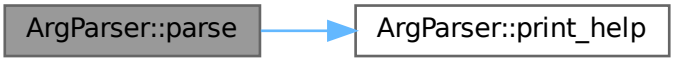
Exceptions

std::runtime_error	If a required argument is missing or if a non-flag argument lacks an associated value.

Note

If the user passes "--help" or "-h", the help message is printed and the program terminates.

Here is the call graph for this function:

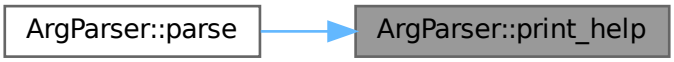


3.1.3.10 print_help()

```
void ArgParser::print_help ( )  [inline]
```

Prints the help message to stdout.

Displays usage information based on the defined arguments and flags, including any default values if provided. Here is the caller graph for this function:



3.1.3.11 set()

```
void ArgParser::set (
            const std::string & name,
            const std::string & value )  [inline]
```

Manually sets the value of an argument.

Parameters

name	The name of the argument to set.
value	The value to assign to the argument.

Exceptions

std::runtime_error	If the argument name is unknown.

Note

This can be useful for programmatically overriding argument values after parsing.

The documentation for this class was generated from the following file:

- include/args.hpp

3.2 ArgParser::Argument Struct Reference

Represents a single command-line argument or flag.

```
#include <args.hpp>
```

Public Attributes

- std::string name
- std::string description
- bool required
- bool is_flag
- std::optional< std::string > default_value
- std::optional< std::string > value

3.2.1 Detailed Description

Represents a single command-line argument or flag.

3.2.2 Member Data Documentation

3.2.2.1 default_value

`ArgParser::Argument::default_value`

Optional default value for this argument if not supplied.

3.2.2.2 description

`ArgParser::Argument::description`

A short description of the argument's purpose.

3.2.2.3 is_flag

`ArgParser::Argument::is_flag`

Indicates whether this argument is a flag (no value needed).

3.2.2.4 name

`ArgParser::Argument::name`

The name of the argument (e.g., "--input").

3.2.2.5 required

`ArgParser::Argument::required`

Indicates whether this argument is required.

3.2.2.6 value

`ArgParser::Argument::value`

The actual value assigned to this argument after parsing.

The documentation for this struct was generated from the following file:

- include/args.hpp

3.3 built_file Struct Reference

Structure holding transformed transaction data for mining.

`#include <build.hpp>`

Public Member Functions

- built_file ()
  Default constructor.
- ~built_file ()
  Destructor.

Public Attributes

- std::map< uint32_t, uint32_t > ordered_twu
  Items ordered by transaction-weighted utility.
- std::vector< std::pair< uint32_t, uint32_t > > ordered_twu_vector
  Vector version of ordered_twu for fast access.
- std::unordered_map< uint32_t, uint32_t > item_to_itemID
  Maps original item IDs to internal IDs.
- std::unordered_map< uint32_t, uint32_t > itemID_to_item
  Maps internal item IDs back to original item IDs.
- size_t transaction_count
  Number of transactions.
- size_t total_items
  Total number of items after pruning.
- size_t max_transaction_size
  Size of the largest transaction.
- std::vector< std::vector< Item > > transactions
  List of compacted transactions.
- std::vector< uint32_t > primary
  Items considered primary (for mining root).

3.3.1 Detailed Description

Structure holding transformed transaction data for mining.

Stores item mappings, transaction utility info, and compacted transactions.

3.3.2 Constructor & Destructor Documentation

3.3.2.1 built_file()

```
built_file::built_file ( )  [inline]
```

Default constructor.

Initializes counts and sizes to zero.

3.3.2.2 ~built_file()

```
built_file::~built_file ( )  [inline]
```

Destructor.

Clears internal mappings and vectors.

3.3.3 Member Data Documentation

3.3.3.1 item_to_itemID

```
std::unordered_map<uint32_t, uint32_t> built_file::item_to_itemID
```

Maps original item IDs to internal IDs.

3.3.3.2 itemID_to_item

```
std::unordered_map<uint32_t, uint32_t> built_file::itemID_to_item
```

Maps internal item IDs back to original item IDs.

3.3.3.3 max_transaction_size

```
size_t built_file::max_transaction_size
```

Size of the largest transaction.

3.3.3.4 ordered_twu

```
std::map<uint32_t, uint32_t> built_file::ordered_twu
```

Items ordered by transaction-weighted utility.

3.3.3.5 ordered_twu_vector

```
std::vector<std::pair<uint32_t, uint32_t> > built_file::ordered_twu_vector
```

Vector version of ordered_twu for fast access.

3.3.3.6 primary

```
std::vector<uint32_t> built_file::primary
```

Items considered primary (for mining root).

3.3.3.7 total_items

```
size_t built_file::total_items
```

Total number of items after pruning.

3.3.3.8 transaction_count

```
size_t built_file::transaction_count
```

Number of transactions.

3.3.3.9 transactions

```
std::vector<std::vector<Item> > built_file::transactions
```

List of compacted transactions.

The documentation for this struct was generated from the following file:

- include/build.hpp

3.4 Item Struct Reference

```
#include <data_types.hpp>
```

Public Attributes

- unsigned int key
- unsigned int util

Static Public Attributes

- static const unsigned int EMPTY_KEY = 0
- static const unsigned int EMPTY_UTIL = 0

3.4.1 Member Data Documentation

3.4.1.1 EMPTY_KEY

```
const unsigned int Item::EMPTY_KEY = 0  [static]
```

3.4.1.2 EMPTY_UTIL

```
const unsigned int Item::EMPTY_UTIL = 0  [static]
```

3.4.1.3 key

```
unsigned int Item::key
```

3.4.1.4 util

```
unsigned int Item::util
```

The documentation for this struct was generated from the following file:

- include/data_types.hpp

3.5 parsed_file Struct Reference

Holds parsed file contents and associated structures.

```
#include <file.hpp>
```

Public Attributes

- std::vector< std::vector< std::pair< std::vector< uint32_t >, std::vector< uint32_t > > > > key_value_pairs
  Nested structure of parsed key/value pairs.
- std::unordered_map< uint32_t, uint32_t > twu
  A map of item IDs to their total weighted utility (TWU).

3.5.1 Detailed Description

Holds parsed file contents and associated structures.

Represents a parsed file with structured key-value pairs and utility data.

This structure contains:

- A nested vector of key-value pairs, where each key and value is a vector of uint32_t.
- A map of total weighted utility (TWU) values for items.

3.5.2 Member Data Documentation

3.5.2.1 key_value_pairs

```
std::vector<std::vector<std::pair<std::vector<uint32_t>, std::vector<uint32_t> > > > parsed↩
_file::key_value_pairs
```

Nested structure of parsed key/value pairs.

Each element represents: vector< pair< vector<keys>, vector<values> > >

3.5.2.2 twu

```
std::unordered_map<uint32_t, uint32_t> parsed_file::twu
```

A map of item IDs to their total weighted utility (TWU).

The documentation for this struct was generated from the following file:

- include/file.hpp

3.6 raw_file Struct Reference

Holds raw file data and associated metadata.

```
#include <file.hpp>
```

Public Member Functions

- raw_file ()
  Constructor initializes defaults.
- void close ()
  Frees the file data buffer, if allocated.

Public Attributes

- char * data
  Pointer to the file's data loaded into memory.
- size_t size_bytes
  Total size of the file in bytes.
- size_t submitted_bytes
  Number of bytes submitted to io_uring (if using async I/O).
- size_t retrieved_bytes
  Number of bytes retrieved from io_uring (if using async I/O).
- size_t processed_bytes
  Number of bytes already processed from the data buffer.
- std::vector< std::pair< size_t, size_t > > retrieved_indices
  Offsets or indices of chunks retrieved via io_uring.
- int fd
  File descriptor for the open file (could be valid or -1 if invalid).

3.6.1 Detailed Description

Holds raw file data and associated metadata.

Represents a raw file loaded into memory.

This structure contains:

- The file's data in memory.
- Metadata about the file, including size, submitted, retrieved, and processed bytes.
- A vector of retrieved indices for tracking processed data.
- The file descriptor for the opened file.

Note

The `close` method should be called to free allocated memory.

3.6.2 Constructor & Destructor Documentation

3.6.2.1 raw_file()

```
raw_file::raw_file ( )  [inline]
```

Constructor initializes defaults.

3.6.3 Member Function Documentation

3.6.3.1 close()

```
void raw_file::close ( )  [inline]
```

Frees the file data buffer, if allocated.

3.6.4 Member Data Documentation

3.6.4.1 data

```
char* raw_file::data
```

Pointer to the file's data loaded into memory.

3.6.4.2 fd

```
int raw_file::fd
```

File descriptor for the open file (could be valid or -1 if invalid).

3.6.4.3 processed_bytes

```
size_t raw_file::processed_bytes
```

Number of bytes already processed from the data buffer.

3.6.4.4 retrieved_bytes

```
size_t raw_file::retrieved_bytes
```

Number of bytes retrieved from io_uring (if using async I/O).

3.6.4.5 retrieved_indices

```
std::vector<std::pair<size_t, size_t> > raw_file::retrieved_indices
```

Offsets or indices of chunks retrieved via io_uring.

3.6.4.6 size_bytes

```
size_t raw_file::size_bytes
```

Total size of the file in bytes.

3.6.4.7 submitted_bytes

```
size_t raw_file::submitted_bytes
```

Number of bytes submitted to io_uring (if using async I/O).

The documentation for this struct was generated from the following file:

- include/file.hpp

3.7 Timer Class Reference

Tracks execution time and memory usage between labeled checkpoints.

```
#include <timer.hpp>
```

Public Member Functions

- void recordPoint (const std::string &label)

  Records a timestamp and current memory with a label.
- void printRecords () const

  Prints all recorded time points and memory usage.
- int getTimeDifference (const std::string &label1, const std::string &label2) const

  Gets time difference between two labeled points.

3.7.1 Detailed Description

Tracks execution time and memory usage between labeled checkpoints.

3.7.2 Member Function Documentation

3.7.2.1 getTimeDifference()

```
int Timer::getTimeDifference (
            const std::string & label1,
            const std::string & label2 ) const  [inline]
```

Gets time difference between two labeled points.

Parameters

label1	First label.
label2	Second label.

Returns

Time in milliseconds.

Exceptions

std::runtime_error	if label(s) not found.

3.7.2.2 printRecords()

```
void Timer::printRecords ( ) const  [inline]
```

Prints all recorded time points and memory usage.

Shows differences between points and total elapsed time.

3.7.2.3 recordPoint()

```
void Timer::recordPoint (
            const std::string & label )  [inline]
```

Records a timestamp and current memory with a label.

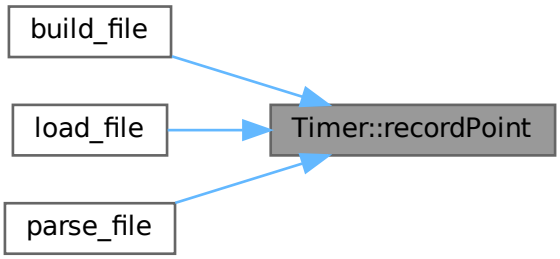
Parameters

label	Label to identify the checkpoint.

Here is the call graph for this function:



Here is the caller graph for this function:



The documentation for this class was generated from the following file:

- include/timer.hpp

3.8 TimeRecord Struct Reference

Stores timestamp and memory usage for a labeled point in time.

`#include <timer.hpp>`

Public Attributes

- std::string label

  Label for the time point.
- std::chrono::high_resolution_clock::time_point timestamp

  Time recorded.
- size_t rss

  Resident Set Size (memory in KB).

3.8.1 Detailed Description

Stores timestamp and memory usage for a labeled point in time.

3.8.2 Member Data Documentation

3.8.2.1 label

```
std::string TimeRecord::label
```

Label for the time point.

3.8.2.2 rss

```
size_t TimeRecord::rss
```

Resident Set Size (memory in KB).

3.8.2.3 timestamp

```
std::chrono::high_resolution_clock::time_point TimeRecord::timestamp
```

Time recorded.

The documentation for this struct was generated from the following file:

- include/timer.hpp

Chapter 4

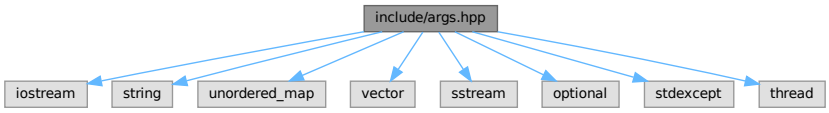
File Documentation

4.1 include/args.hpp File Reference

Lightweight C++ command-line argument parser.

```
#include <iostream>
#include <string>
#include <unordered_map>
#include <vector>
#include <sstream>
#include <optional>
#include <stdexcept>
#include <thread>
```

Include dependency graph for args.hpp:



Classes

- class ArgParser

  Simple command-line argument parser.
- struct ArgParser::Argument

  Represents a single command-line argument or flag.

4.1.1 Detailed Description

Lightweight C++ command-line argument parser.

This file declares the ArgParser class, which supports defining and parsing flags, required arguments, and optional arguments with default values. It also provides type-safe getters, validation, and an auto-generated help message.

The file also includes a helper function, getSeparator, to convert string representations of separators (e.g., "\\s", "\\t") into their corresponding character values.

Note

The parser exits with an error if arguments are invalid or required ones are missing.

4.2 args.hpp

Go to the documentation of this file.

```
00001
00016 #pragma once
00017
00018 #include <iostream>
00019 #include <string>
00020 #include <unordered_map>
00021 #include <vector>
00022 #include <sstream>
00023 #include <optional>
00024 #include <stdexcept>
00025 #include <thread>
00026
00036 static char getSeparator(const std::string &sepStr)
00037 {
00038     if (sepStr == "\\s")
00039         return ' ';
00040     if (sepStr == "\\t")
00041         return '\t';
00042     if (sepStr == "\\n")
00043         return '\n';
00044     if (sepStr == ",")
00045         return ',';
00046     if (sepStr == ";")
00047         return ';';
00048     if (sepStr == "|")
00049         return '|';
00050
00051     // Default separator (fallback to space)
00052     std::cerr << "Unknown separator: " << sepStr << ". Exiting.\n";
00053     std::exit(1);
00054 }
00055
00064 class ArgParser
00065 {
00066 public:
00089     struct Argument
00090     {
00091         std::string name;
00092         std::string description;
00093         bool required;
00094         bool is_flag;
00095         std::optional<std::string> default_value;
00096         std::optional<std::string> value;
00097     };
00098
00099 private:
00100     std::unordered_map<std::string, Argument> args;
00101     std::vector<std::string> positional_args;
00102     std::string program_name;
00103
00104 public:
00111     explicit ArgParser(std::string prog_name) : program_name(std::move(prog_name)) {}
00112
00124     void add_argument(const std::string &name,
00125                       const std::string &desc,
00126                       bool required = false,
00127                       std::optional<std::string> default_val = std::nullopt)
00128     {
00129         args[name] = {name, desc, required, false, default_val, std::nullopt};
00130     }
00131
00141     void add_flag(const std::string &name, const std::string &desc)
00142     {
00143         args[name] = {name, desc, false, true, std::nullopt, std::nullopt};
00144     }
00145
00157     void parse(int argc, char *argv[])
00158     {
00159         try
00160         {
00161             if (argc > 0)
00162                 program_name = argv[0];
00163
00164             for (int i = 1; i < argc; ++i)
00165             {
00166                 std::string arg = argv[i];
00167
00168                 // Check if the argument matches a known flag or argument
00169                 if (args.find(arg) != args.end())
00170                 {
00171                     // It's a flag
00172                     if (args[arg].is_flag)
```

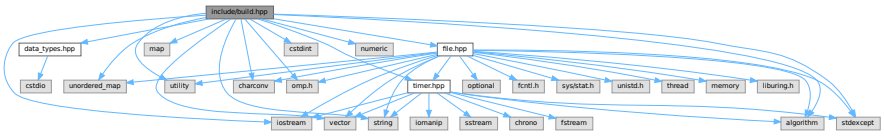
```
00173                     {
00174                         args[arg].value = "true";
00175                     }
00176                     // It's a regular argument
00177                     else if (i + 1 < argc)
00178                     {
00179                         args[arg].value = argv[++i];
00180                     }
00181                     // Value missing
00182                     else
00183                     {
00184                         throw std::runtime_error("Missing value for argument: " + arg);
00185                     }
00186                 }
00187                 else if (arg == "--help" || arg == "-h")
00188                 {
00189                     print_help();
00190                     std::exit(0);
00191                 }
00192                 else
00193                 {
00194                     // Anything that doesn't match a known argument/flag is treated as positional
00195                     positional_args.push_back(arg);
00196                 }
00197             }
00198 
00199             // Validate required arguments
00200             for (const auto &[key, arg] : args)
00201             {
00202                 if (arg.required && !arg.value.has_value())
00203                 {
00204                     throw std::runtime_error("Missing required argument: " + key);
00205                 }
00206             }
00207 
00208             // Apply default values where needed
00209             for (auto &[key, arg] : args)
00210             {
00211                 if (!arg.value.has_value() && arg.default_value.has_value())
00212                 {
00213                     arg.value = arg.default_value;
00214                 }
00215             }
00216         }
00217         catch (const std::exception &e)
00218         {
00219             std::cerr « "Error: " « e.what() « "\n\n";
00220             print_help();
00221             std::exit(1);
00222         }
00223     }
00224 
00235     std::string get(const std::string &name)
00236     {
00237         if (args.find(name) == args.end())
00238             throw std::runtime_error("Unknown argument: " + name);
00239         return args[name].value.value_or("");
00240     }
00241 
00250     bool get_bool(const std::string &name)
00251     {
00252         if (args.find(name) == args.end())
00253             throw std::runtime_error("Unknown argument: " + name);
00254         return args[name].value.has_value() && args[name].value.value() == "true";
00255     }
00256 
00265     uint32_t get_uint32(const std::string &name)
00266     {
00267         return std::stoul(get(name));
00268     }
00269 
00278     uint64_t get_uint64(const std::string &name)
00279     {
00280         return std::stoull(get(name));
00281     }
00282 
00292     char get_separator(const std::string &name)
00293     {
00294         return getSeparator(get(name));
00295     }
00296 
00305     bool flag_set(const std::string &name)
00306     {
00307         if (args.find(name) == args.end())
00308             throw std::runtime_error("Unknown flag: " + name);
00309         return args[name].value.has_value();
00310     }
```

```
00311
00318     void print_help()
00319     {
00320         std::cout « "Usage: " « program_name « " [options]\n\n"
00321                  « "Options:\n";
00322         for (const auto &[key, arg] : args)
00323         {
00324             std::cout « "  " « key;
00325             if (!arg.is_flag)
00326                 std::cout « " <value>";
00327             std::cout « "\t" « arg.description;
00328             if (arg.default_value.has_value())
00329                 std::cout « " (default: " « arg.default_value.value() « ")";
00330             std::cout « "\n";
00331         }
00332         std::cout « "  --help, -h\tShow this help message\n";
00333     }
00334
00346     void set(const std::string &name, const std::string &value)
00347     {
00348         if (args.find(name) == args.end())
00349             throw std::runtime_error("Unknown argument: " + name);
00350         args[name].value = value;
00351     }
00352 };
```

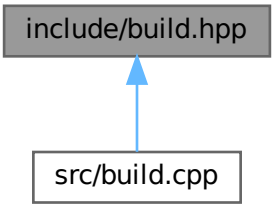
4.3 include/build.hpp File Reference

```
#include <iostream>
#include <vector>
#include <string>
#include <unordered_map>
#include <map>
#include <utility>
#include <charconv>
#include <stdexcept>
#include <omp.h>
#include <cstdint>
#include <algorithm>
#include <numeric>
#include "data_types.hpp"
#include "timer.hpp"
#include "file.hpp"
```

Include dependency graph for build.hpp:



This graph shows which files directly or indirectly include this file:



Classes

- struct built_file

  Structure holding transformed transaction data for mining.

Functions

- built_file build_file (parsed_file &pf, int threads, Timer &timer, uint64_t min_utility)

  Builds a structured and filtered transaction dataset from parsed input.

4.3.1 Function Documentation

4.3.1.1 build_file()

```
built_file build_file (
            parsed_file & pf,
            int threads,
            Timer & timer,
            uint64_t min_utility )
```

Builds a structured and filtered transaction dataset from parsed input.

Transforms a parsed_file into a built_file by filtering low-utility items, remapping item IDs, and computing statistics like TWU and max transaction size.

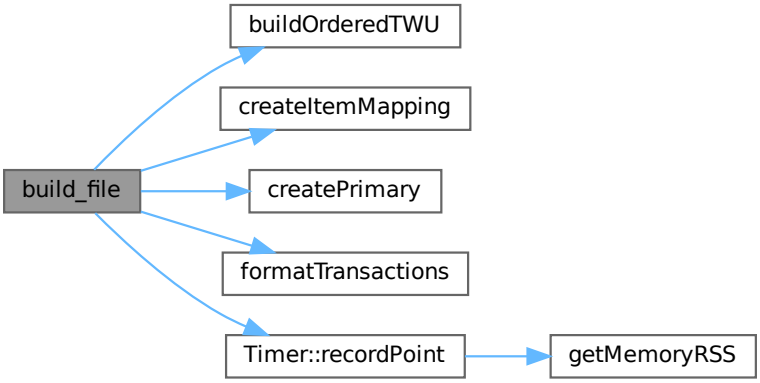
Parameters

pf	The parsed file input.
threads	Number of threads to use (OpenMP enabled).
timer	A Timer object for benchmarking.
min_utility	The minimum utility threshold to keep an item.

Returns

A built_file structure containing the processed transaction data.

Here is the call graph for this function:



4.4 build.hpp

Go to the documentation of this file.

```
00001
00009  #pragma once
00010
00011  #include <iostream>
00012  #include <vector>
00013  #include <string>
00014  #include <unordered_map>
00015  #include <map>
00016  #include <utility>
00017  #include <charconv>
00018  #include <stdexcept>
00019  #include <omp.h>
00020  #include <cstdint>
00021  #include <algorithm>
00022  #include <numeric>
00023  #include "data_types.hpp"
00024  #include "timer.hpp"
00025  #include "file.hpp"
00026
00033  struct built_file
00034  {
00035      std::map<uint32_t, uint32_t> ordered_twu;
00036      std::vector<std::pair<uint32_t, uint32_t» ordered_twu_vector;
00037
00038      std::unordered_map<uint32_t, uint32_t> item_to_itemID;
00039      std::unordered_map<uint32_t, uint32_t> itemID_to_item;
00040
00041      size_t transaction_count;
00042      size_t total_items;
00043      size_t max_transaction_size;
00044
00045      std::vector<std::vector<Item» transactions;
00046      std::vector<uint32_t> primary;
00047
00053      built_file()
00054      {
00055          transaction_count = 0;
00056          max_transaction_size = 0;
00057          total_items = 0;
00058      }
00059
```

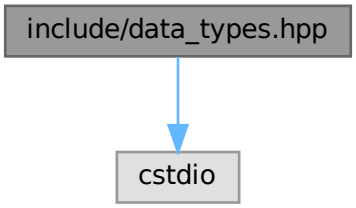
```
00065       ~built_file()
00066       {
00067           ordered_twu.clear();
00068           ordered_twu_vector.clear();
00069           item_to_itemID.clear();
00070           itemID_to_item.clear();
00071       }
00072   };
00073
00086   built_file build_file(parsed_file &pf, int threads, Timer &timer, uint64_t min_utility);
00087
```

4.5 include/copy.cuh File Reference

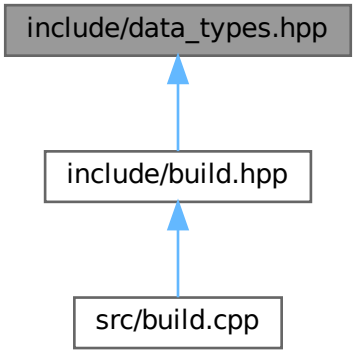
4.6 include/data_types.cuh File Reference

4.7 include/data_types.hpp File Reference

`#include <cstdio>`
Include dependency graph for data_types.hpp:



This graph shows which files directly or indirectly include this file:



Classes

- struct Item

4.8 data_types.hpp

Go to the documentation of this file.

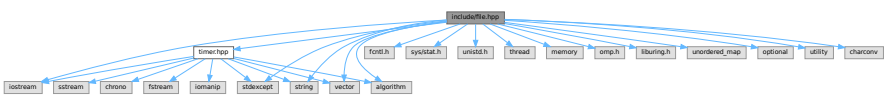
```
00001 #pragma once
00002
00003 #include <cstdio>
00004
00005 // Represents an item with key and utility.
00006 struct Item
00007 {
00008     unsigned int key;
00009     unsigned int util;
00010
00011     static const unsigned int EMPTY_KEY = 0;
00012     static const unsigned int EMPTY_UTIL = 0;
00013 };
00014
```

4.9 include/file.hpp File Reference

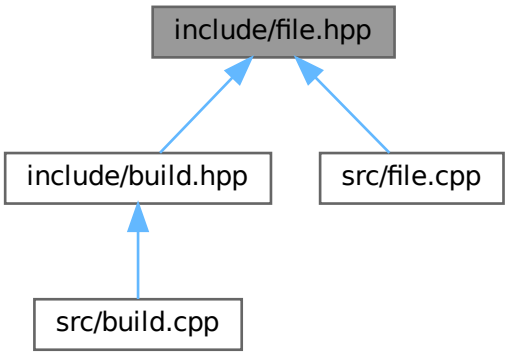
Provides structures and functions for handling raw and parsed file data, including file loading, parsing, and utility calculations.

```
#include <iostream>
#include <fcntl.h>
#include <sys/stat.h>
#include <unistd.h>
#include <thread>
#include <memory>
#include "timer.hpp"
#include <vector>
#include <string>
#include <omp.h>
#include <liburing.h>
#include <unordered_map>
#include <optional>
#include <stdexcept>
#include <algorithm>
#include <utility>
#include <charconv>
```

Include dependency graph for file.hpp:



This graph shows which files directly or indirectly include this file:



Classes

- struct raw_file

  Holds raw file data and associated metadata.
- struct parsed_file

  Holds parsed file contents and associated structures.

Enumerations

- enum ParseState { PARSE_ITEMS , PARSE_TOTAL_UTILITY , PARSE_UTILITIES }

  Enum representing different parsing states for parse_file().

Functions

- raw_file load_file (std::string filename, int threads, Timer &timer)

  Loads a file into a raw_file structure, optionally using multiple threads.
- parsed_file parse_file (raw_file &file, int threads, const char separator, Timer &timer)

  Parses a raw_file into a structured format (parsed_file).

4.9.1 Detailed Description

Provides structures and functions for handling raw and parsed file data, including file loading, parsing, and utility calculations.

This header defines the following:

- `raw_file`: A structure to manage raw file data and its associated metadata.
- `parsed_file`: A structure to store parsed key-value pairs and utility data.
- `load_file`: A function to load a file into memory using io_uring for asynchronous I/O.
- `parse_file`: A function to parse raw file data into structured key-value pairs.
- `ParseState`: An enumeration to represent the state of parsing.

4.9.2 Enumeration Type Documentation

4.9.2.1 ParseState

```
enum ParseState
```

Enum representing different parsing states for parse_file().

Represents the state of parsing a file.

Possible states:

- PARSE_ITEMS: Parsing items from the file.
- PARSE_TOTAL_UTILITY: Parsing the total utility value.
- PARSE_UTILITIES: Parsing individual utilities.

Enumerator

PARSE_ITEMS	Parsing item keys.
PARSE_TOTAL_UTILITY	Parsing total utility field.
PARSE_UTILITIES	Parsing per-item utilities.

4.9.3 Function Documentation

4.9.3.1 load_file()

```
raw_file load_file (
            std::string filename,
            int threads,
            Timer & timer )
```

Loads a file into a raw_file structure, optionally using multiple threads.

Parameters

filename	Path to the file to be loaded.
threads	Number of threads to use for loading (if applicable).
timer	Reference to a timing object used for measuring performance.

Returns

A fully populated raw_file struct.

Loads a file into a raw_file structure, optionally using multiple threads.

Parameters

filename	The path to the file to be loaded.
threads	The number of threads to use for loading.
timer	A reference to a Timer object for measuring performance.

Returns

A `raw_file` structure containing the loaded file data and metadata.

Exceptions

std::runtime_error	If the file cannot be opened or read.

Here is the call graph for this function:



4.9.3.2 parse_file()

```
parsed_file parse_file (
            raw_file & file,
            int threads,
            const char separator,
            Timer & timer )
```

Parses a raw_file into a structured format (parsed_file).

Parameters

file	The raw_file to be parsed.
threads	Number of threads to be used during parsing.
separator	The character used to separate fields.
timer	Reference to a timing object for performance measurement.

Returns

A parsed_file containing structured data.

Parses a raw_file into a structured format (parsed_file).

Parameters

file	A reference to the `raw_file` structure containing the file data.
threads	The number of threads to use for parsing.
separator	The character used to separate items in the file.
timer	A reference to a Timer object for measuring performance.

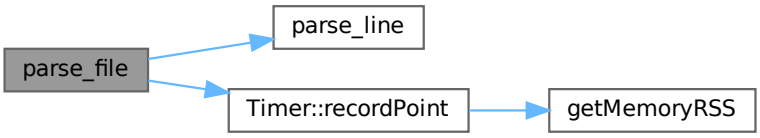
Returns

A `parsed_file` structure containing the parsed data and utility information.

Exceptions

std::invalid_argument	If the file data is invalid or cannot be parsed.

Here is the call graph for this function:



4.10 file.hpp

Go to the documentation of this file.

```
00001
00070 #pragma once
00071
00072 #include <iostream>
00073 #include <fcntl.h>
00074 #include <sys/stat.h>
00075 #include <unistd.h>
00076 #include <thread>
00077 #include <memory>
00078 #include "timer.hpp"
00079 #include <vector>
00080 #include <string>
00081 #include <omp.h>
00082 #include <liburing.h>
00083 #include <unordered_map>
00084 #include <optional>
00085 #include <stdexcept>
00086 #include <algorithm>
00087 #include <utility>
00088 #include <charconv>
00089
00093 struct raw_file
00094 {
00096     char *data;
00097
00099     size_t size_bytes;
00100
00102     size_t submitted_bytes;
00103
00105     size_t retrieved_bytes;
```

```
00106
00108     size_t processed_bytes;
00109
00111     std::vector<std::pair<size_t, size_t» retrieved_indices;
00112
00114     int fd;
00115
00119     raw_file() : data(nullptr), size_bytes(0), submitted_bytes(0),
00120                  retrieved_bytes(0), processed_bytes(0), fd(-1) {}
00121
00125     void close()
00126     {
00127         if (data != nullptr)
00128             free(data);
00129     }
00130 };
00131
00139 raw_file load_file(std::string filename, int threads, Timer &timer);
00140
00144 struct parsed_file
00145 {
00152     std::vector<std::vector<std::pair<std::vector<uint32_t>, std::vector<uint32_t»» key_value_pairs;
00153
00157     std::unordered_map<uint32_t, uint32_t> twu;
00158 };
00159
00163 enum ParseState {
00164     PARSE_ITEMS,
00165     PARSE_TOTAL_UTILITY,
00166     PARSE_UTILITIES
00167 };
00168
00177 parsed_file parse_file(raw_file &file, int threads, const char separator, Timer &timer);
```

4.11 include/memory.cuh File Reference

4.12 include/mine.cuh File Reference

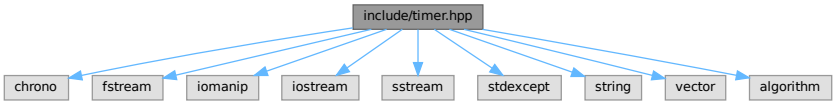
4.13 include/stack.cuh File Reference

4.14 include/timer.hpp File Reference

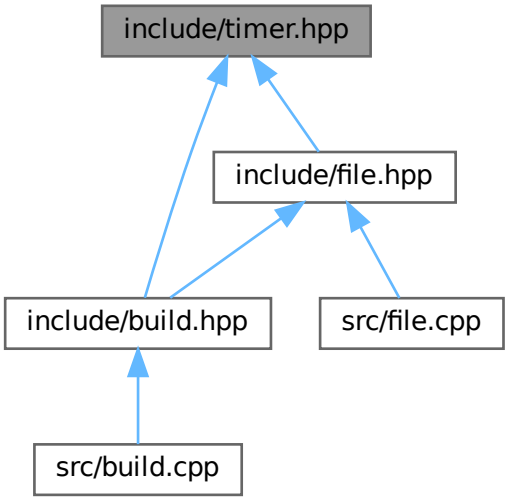
Simple timer and memory profiler for labeled code segments.

```
#include <chrono>
#include <fstream>
#include <iomanip>
#include <iostream>
#include <sstream>
#include <stdexcept>
#include <string>
#include <vector>
#include <algorithm>
```

Include dependency graph for timer.hpp:



This graph shows which files directly or indirectly include this file:



Classes

- struct TimeRecord

  Stores timestamp and memory usage for a labeled point in time.
- class Timer

  Tracks execution time and memory usage between labeled checkpoints.

Functions

- size_t getMemoryRSS ()

  Reads the current memory RSS from /proc/self/status.

4.14.1 Detailed Description

Simple timer and memory profiler for labeled code segments.

4.14.2 Function Documentation

4.14.2.1 getMemoryRSS()

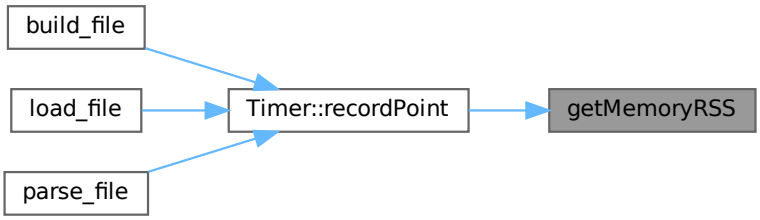
```
size_t getMemoryRSS ( )  [inline]
```

Reads the current memory RSS from /proc/self/status.

Returns

Memory usage in kilobytes.

Here is the caller graph for this function:



4.15 timer.hpp

Go to the documentation of this file.

```
00001
00006  #pragma once
00007
00008  #include <chrono>
00009  #include <fstream>
00010  #include <iomanip>
00011  #include <iostream>
00012  #include <sstream>
00013  #include <stdexcept>
00014  #include <string>
00015  #include <vector>
00016  #include <algorithm>
00017
00022  struct TimeRecord
00023  {
00024      std::string label;
00025      std::chrono::high_resolution_clock::time_point timestamp;
00026      size_t rss;
00027  };
00028
00033  inline size_t getMemoryRSS()
00034  {
00035      std::ifstream status_file("/proc/self/status");
00036      std::string line;
00037      while (std::getline(status_file, line))
00038      {
00039          if (line.substr(0, 6) == "VmRSS:")
00040          {
00041              std::istringstream iss(line);
00042              std::string key, unit;
00043              size_t value;
00044              if (iss » key » value » unit)
00045                  return value;
00046          }
00047      }
00048      return 0;
00049  }
00050
00055  class Timer
00056  {
00057  public:
00062      void recordPoint(const std::string &label)
00063      {
00064          TimeRecord rec;
00065          rec.label = label;
00066          rec.timestamp = std::chrono::high_resolution_clock::now();
00067          rec.rss = getMemoryRSS();
00068          records.push_back(rec);
00069      }
```

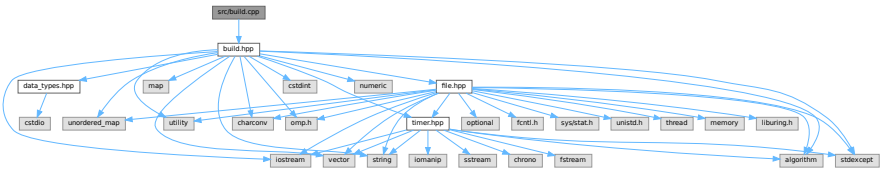
```
00070
00076     void printRecords() const
00077     {
00078         if (records.empty())
00079         {
00080             std::cout « "No records to display." « std::endl;
00081             return;
00082         }
00083
00084         size_t max_label_len = 0;
00085         for (const auto &rec : records)
00086             max_label_len = std::max(max_label_len, rec.label.length());
00087
00088         std::cout « std::left « std::setw(static_cast<int>(max_label_len) + 4) « "Label"
00089                   « std::right « std::setw(12) « "Time (ms)"
00090                   « std::right « std::setw(16) « "Memory (KB)" « std::endl;
00091         std::cout « std::string(max_label_len + 32, '-') « std::endl;
00092
00093         std::cout « std::left « std::setw(static_cast<int>(max_label_len) + 4) « records[0].label
00094                   « std::right « std::setw(12) « 0
00095                   « std::right « std::setw(16) « records[0].rss « std::endl;
00096
00097         for (size_t i = 1; i < records.size(); ++i)
00098         {
00099             auto time_diff = std::chrono::duration_cast<std::chrono::milliseconds>(
00100                                 records[i].timestamp - records[i - 1].timestamp)
00101                                 .count();
00102             auto mem_diff = static_cast<int>(records[i].rss) - static_cast<int>(records[i - 1].rss);
00103             std::cout « std::left « std::setw(static_cast<int>(max_label_len) + 4) « records[i].label
00104                       « std::right « std::setw(12) « time_diff
00105                       « std::right « std::setw(16) « records[i].rss
00106                       « " (" « (mem_diff >= 0 ? "+" : "") « mem_diff « ")" « std::endl;
00107         }
00108
00109         auto total = std::chrono::duration_cast<std::chrono::milliseconds>(
00110                         records.back().timestamp - records.front().timestamp)
00111                         .count();
00112         std::cout « std::string(max_label_len + 32, '-') « std::endl;
00113         std::cout « std::left « std::setw(static_cast<int>(max_label_len) + 4) « "Total time"
00114                   « std::right « std::setw(12) « total « " ms" « std::endl;
00115     }
00116
00124     int getTimeDifference(const std::string &label1, const std::string &label2) const
00125     {
00126         auto it1 = std::find_if(records.begin(), records.end(), [&label1](const TimeRecord &rec)
00127                                 { return rec.label == label1; });
00128         auto it2 = std::find_if(records.begin(), records.end(), [&label2](const TimeRecord &rec)
00129                                 { return rec.label == label2; });
00130
00131         if (it1 == records.end() || it2 == records.end())
00132             throw std::runtime_error("One or both labels not found.");
00133
00134         return std::chrono::duration_cast<std::chrono::milliseconds>(
00135                    it2->timestamp - it1->timestamp)
00136             .count();
00137     }
00138
00139 private:
00140     std::vector<TimeRecord> records;
00141 };
00142
```

4.16 include/validate.cuh File Reference

4.17 src/build.cpp File Reference

```
#include "build.hpp"
```

Include dependency graph for build.cpp:



Functions

- void buildOrderedTWU (const parsed_file &pf, uint64_t min_utility, built_file &bf)
- void createItemMapping (built_file &bf)
- void formatTransactions (const parsed_file &pf, int threads, const built_file &bf, std::vector< std::vector< Item > > &transactions, std::unordered_map< uint32_t, uint32_t > &global_twu)
- void createPrimary (built_file &bf, uint64_t min_utility, std::unordered_map< uint32_t, uint32_t > &subtree↩ _utility)
- built_file build_file (parsed_file &pf, int threads, Timer &timer, uint64_t min_utility)

  Builds a structured and filtered transaction dataset from parsed input.

4.17.1 Function Documentation

4.17.1.1 build_file()

```
built_file build_file (
            parsed_file & pf,
            int threads,
            Timer & timer,
            uint64_t min_utility )
```

Builds a structured and filtered transaction dataset from parsed input.

Transforms a parsed_file into a built_file by filtering low-utility items, remapping item IDs, and computing statistics like TWU and max transaction size.

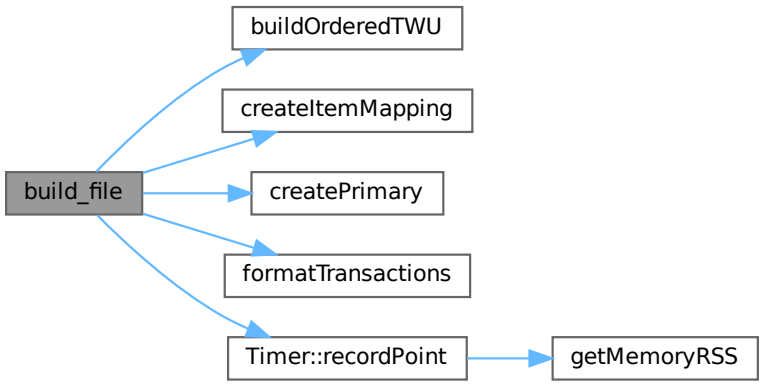
Parameters

pf	The parsed file input.
threads	Number of threads to use (OpenMP enabled).
timer	A Timer object for benchmarking.
min_utility	The minimum utility threshold to keep an item.

Returns

A built_file structure containing the processed transaction data.

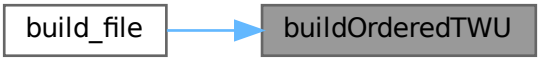
Here is the call graph for this function:



4.17.1.2 buildOrderedTWU()

```
void buildOrderedTWU (
            const parsed_file & pf,
            uint64_t min_utility,
            built_file & bf )
```

Here is the caller graph for this function:



4.17.1.3 createItemMapping()

```
void createItemMapping (
            built_file & bf )
```

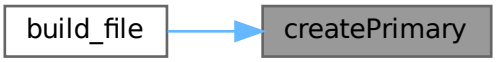
Here is the caller graph for this function:



4.17.1.4 createPrimary()

```
void createPrimary (
            built_file & bf,
            uint64_t min_utility,
            std::unordered_map< uint32_t, uint32_t > & subtree_utility )
```

Here is the caller graph for this function:



4.17.1.5 formatTransactions()

```
void formatTransactions (
            const parsed_file & pf,
            int threads,
            const built_file & bf,
            std::vector< std::vector< Item > > & transactions,
            std::unordered_map< uint32_t, uint32_t > & global_twu )
```

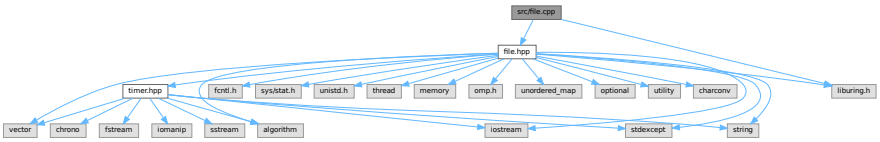
Here is the caller graph for this function:



4.18 src/file.cpp File Reference

```
#include "file.hpp"
#include <liburing.h>
```

Include dependency graph for file.cpp:



Functions

- raw_file load_file (std::string filename, int threads, Timer &timer)

  Loads a file into memory using asynchronous I/O.
- std::tuple< std::vector< uint32_t >, std::vector< uint32_t >, uint32_t > parse_line (const char *data, size_t start, size_t end, const char separator)
- parsed_file parse_file (raw_file &file, int threads, const char separator, Timer &timer)

  Parses a raw file into structured key-value pairs and calculates utility data.

4.18.1 Function Documentation

4.18.1.1 load_file()

```
raw_file load_file (
            std::string filename,
            int threads,
            Timer & timer )
```

Loads a file into memory using asynchronous I/O.

Loads a file into a raw_file structure, optionally using multiple threads.

Parameters

filename	The path to the file to be loaded.
threads	The number of threads to use for loading.
timer	A reference to a Timer object for measuring performance.

Returns

A raw_file structure containing the loaded file data and metadata.

Exceptions

std::runtime_error	If the file cannot be opened or read.

Here is the call graph for this function:



4.18.1.2 parse_file()

```
parsed_file parse_file (
            raw_file & file,
            int threads,
            const char separator,
            Timer & timer )
```

Parses a raw file into structured key-value pairs and calculates utility data.

Parses a raw_file into a structured format (parsed_file).

Parameters

file	A reference to the `raw_file` structure containing the file data.
threads	The number of threads to use for parsing.
separator	The character used to separate items in the file.
timer	A reference to a Timer object for measuring performance.

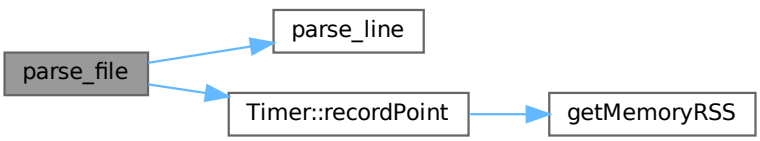
Returns

A `parsed_file` structure containing the parsed data and utility information.

Exceptions

std::invalid_argument	If the file data is invalid or cannot be parsed.

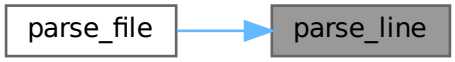
Here is the call graph for this function:



4.18.1.3 parse_line()

```
std::tuple< std::vector< uint32_t >, std::vector< uint32_t >, uint32_t > parse_line (
            const char * data,
            size_t start,
            size_t end,
            const char separator )
```

Here is the caller graph for this function:



Index

~built_file
    built_file, 14

add_argument
    ArgParser, 6
add_flag
    ArgParser, 6
ArgParser, 5
    add_argument, 6
    add_flag, 6
    ArgParser, 6
    flag_set, 7
    get, 7
    get_bool, 8
    get_separator, 8
    get_uint32, 9
    get_uint64, 10
    parse, 10
    print_help, 11
    set, 11
ArgParser::Argument, 12
    default_value, 13
    description, 13
    is_flag, 13
    name, 13
    required, 13
    value, 13

build.cpp
    build_file, 41
    buildOrderedTWU, 42
    createItemMapping, 42
    createPrimary, 43
    formatTransactions, 43
build.hpp
    build_file, 29
build_file
    build.cpp, 41
    build.hpp, 29
buildOrderedTWU
    build.cpp, 42
built_file, 13
    ~built_file, 14
    built_file, 14
    item_to_itemID, 15
    itemID_to_item, 15
    max_transaction_size, 15
    ordered_twu, 15
    ordered_twu_vector, 15
    primary, 15
    total_items, 15
    transaction_count, 15
    transactions, 16

close
    raw_file, 19
createItemMapping
    build.cpp, 42
createPrimary
    build.cpp, 43

data
    raw_file, 19
default_value
    ArgParser::Argument, 13
description
    ArgParser::Argument, 13

EMPTY_KEY
    Item, 16
EMPTY_UTIL
    Item, 16

fd
    raw_file, 19
file.cpp
    load_file, 44
    parse_file, 45
    parse_line, 45
file.hpp
    load_file, 34
    parse_file, 35
    PARSE_ITEMS, 34
    PARSE_TOTAL_UTILITY, 34
    PARSE_UTILITIES, 34
    ParseState, 34
flag_set
    ArgParser, 7
formatTransactions
    build.cpp, 43

get
    ArgParser, 7
get_bool
    ArgParser, 8
get_separator
    ArgParser, 8
get_uint32
    ArgParser, 9
get_uint64
    ArgParser, 10

getMemoryRSS
timer.hpp, 38
getTimeDifference
Timer, 21

include/args.hpp, 25, 26
include/build.hpp, 28, 30
include/copy.cuh, 31
include/data_types.cuh, 31
include/data_types.hpp, 31, 32
include/file.hpp, 32, 36
include/memory.cuh, 37
include/mine.cuh, 37
include/stack.cuh, 37
include/timer.hpp, 37, 39
include/validate.cuh, 41
is_flag
ArgParser::Argument, 13
Item, 16
EMPTY_KEY, 16
EMPTY_UTIL, 16
key, 16
util, 16
item_to_itemID
built_file, 15
itemID_to_item
built_file, 15

key
Item, 16
key_value_pairs
parsed_file, 17

label
TimeRecord, 23
load_file
file.cpp, 44
file.hpp, 34

max_transaction_size
built_file, 15

name
ArgParser::Argument, 13

ordered_twu
built_file, 15
ordered_twu_vector
built_file, 15

parse
ArgParser, 10
parse_file
file.cpp, 45
file.hpp, 35
PARSE_ITEMS
file.hpp, 34
parse_line
file.cpp, 45
PARSE_TOTAL_UTILITY
file.hpp, 34
PARSE_UTILITIES
file.hpp, 34
parsed_file, 17
key_value_pairs, 17
twu, 17
ParseState
file.hpp, 34
primary
built_file, 15
print_help
ArgParser, 11
printRecords
Timer, 21
processed_bytes
raw_file, 19

raw_file, 18
close, 19
data, 19
fd, 19
processed_bytes, 19
raw_file, 19
retrieved_bytes, 20
retrieved_indices, 20
size_bytes, 20
submitted_bytes, 20
recordPoint
Timer, 21
required
ArgParser::Argument, 13
retrieved_bytes
raw_file, 20
retrieved_indices
raw_file, 20
rss
TimeRecord, 23

set
ArgParser, 11
size_bytes
raw_file, 20
src/build.cpp, 41
src/file.cpp, 44
submitted_bytes
raw_file, 20

Timer, 20
getTimeDifference, 21
printRecords, 21
recordPoint, 21
timer.hpp
getMemoryRSS, 38
TimeRecord, 22
label, 23
rss, 23
timestamp, 23
timestamp
TimeRecord, 23

total_items
built_file, 15
transaction_count
built_file, 15
transactions
built_file, 16
twu
parsed_file, 17

util
Item, 16

value
ArgParser::Argument, 13